Marketplace NFT décentralisée

Rapport technique et support de présentation

Smart contracts Solidity (ERC-721) • Hardhat • OpenZeppelin • React / Vite • MetaMask • IPFS (Pinata)

Projet universitaire – Data Engineering & Decision Systems

Version du rapport : étapes 1 à 8 réalisées

1. Résumé (le pitch en 30 secondes)

Idée : une version simplifiée d'OpenSea. N'importe qui peut créer un NFT, le mettre en vente en ETH et l'acheter, sans intermédiaire de confiance : ce sont deux **smart contracts** qui garantissent la propriété, le prix et le paiement.

- **MyNFT** : le registre de propriété (standard ERC-721). Chaque NFT pointe vers son image et ses informations stockées sur IPFS.
- **NFTMarketplace** : la salle de vente. Vente à prix fixe, achat **atomique** (NFT et paiement passent ensemble ou rien ne se passe), annulation possible.
- **Frontend React** : connexion MetaMask, création, vente, achat, historique, détection du réseau.
- **Sécurité** : OpenZeppelin, protection contre la réentrance, validations à deux niveaux, aucun secret exposé dans le navigateur.
- **Qualité** : 28 tests automatisés sur les contrats, tous réussis.

2. Objectifs et périmètre

Le projet permet à un utilisateur de : connecter MetaMask, créer (mint) un NFT, consulter les NFT en vente, mettre un NFT en vente, acheter, annuler une annonce, et consulter ses NFT et son historique. Il fonctionne sur une blockchain locale (Hardhat) et est prévu pour Ethereum Sepolia (testnet).

Étape	Contenu	État
1	Analyse, architecture, plan	Terminée
2	Initialisation Hardhat + Vite	Terminée
3	Contrat MyNFT	Terminée
4	Contrat NFTMarketplace	Terminée
5	Tests unitaires (28)	Terminée
6	Déploiement local + export ABI/adresses	Terminée
7	Frontend React + MetaMask	Terminée (testé par l'étudiante)
8	Intégration IPFS / Pinata	Terminée (code livré, test en cours)
9	Tests de bout en bout	À faire
10	Mesures de performance + graphiques	À faire
11	Déploiement Sepolia + README final	À faire

Les mesures de performance (latence, gas, débit) et le déploiement Sepolia ne sont pas encore réalisés : ce rapport ne présente donc aucun chiffre de performance.

3. Architecture

Le projet contient deux sous-projets npm : la racine (Hardhat : contrats, tests, scripts) et `frontend/` (React + Vite).

```
nft-marketplace/
 |-- contracts/
 |    |-- MyNFT.sol                (ERC-721 + URI + enumeration)
 |    |-- NFTMarketplace.sol       (annonces, achat, annulation)
 |    '-- mocks/RejectingSeller.sol (contrat de TEST uniquement)
 |-- scripts/deploy.js             (deploie + exporte adresses et ABI)
 |-- test/  MyNFT.test.js, NFTMarketplace.test.js
 '-- frontend/src/
      |-- components/ pages/ hooks/ services/ utils/
      '-- contracts/  deployments.json + ABI (generes par deploy.js)
```

3.1 Rôle de chaque technologie

Technologie	Rôle dans le projet
Solidity 0.8.24	Langage des contrats. Depuis la 0.8, les dépassements arithmétiques font échouer la transaction (pas de SafeMath nécessaire).
OpenZeppelin 5	Implémentations auditées : ERC-721, URIStorage, Enumerable, ReentrancyGuard. On ne réécrit pas les standards.
Hardhat	Compile, teste, simule une blockchain locale (20 comptes de test) et déploie.
Ethers.js v6	Bibliothèque d'accès à la blockchain, dans les scripts et dans le navigateur.
React + Vite	Interface utilisateur, rapide à développer.
MetaMask	Portefeuille : signe les transactions, gère compte et réseau. L'application ne voit jamais la clé privée.
IPFS / Pinata	Stockage décentralisé des images et des métadonnées JSON.

3.2 Ce qui est stocké on-chain et off-chain

On-chain (blockchain)	Off-chain (IPFS)
Propriétaire de chaque tokenId URI des métadonnées (ipfs://CID) Annonces : vendeur et prix Événements (historique)	Image du NFT JSON : nom, description, lien vers l'image

Pourquoi ? Stocker une image sur Ethereum coûterait des centaines de dollars de gas. On ne garde donc on-chain que le strict nécessaire (propriété + référence). Le CID IPFS est l'empreinte du contenu : si le fichier change, le CID change. L'URI enregistrée est donc une preuve d'intégrité du contenu.

3.3 Pourquoi deux contrats séparés ?

- Le NFT reste un **standard réutilisable**, compatible avec MetaMask et les autres places de marché.
- On peut remplacer la marketplace sans toucher aux NFT déjà créés.
- Une responsabilité par contrat : plus simple à tester et à sécuriser.

4. Le contrat MyNFT (en détail)

Rôle : c'est le registre officiel : « le NFT n°5 appartient à tel compte et son contenu est ici ». Il hérite de trois briques OpenZeppelin :

Brique héritée	Ce qu'elle apporte
ERC721	ownerOf, balanceOf, transferFrom, approve, setApprovalForAll : le standard complet de propriété et de transfert.
ERC721URIStorage	Une URI de métadonnées propre à chaque token (tokenURI).
ERC721Enumerable	totalSupply, tokenByIndex, tokenOfOwnerByIndex : permet au frontend de lister tous les NFT ou ceux d'un utilisateur sans base de données.

4.1 Code de la fonction mint

```
uint256 private _nextTokenId = 1;   // les ids commencent a 1

event NFTMinted(uint256 indexed tokenId, address indexed creator, string tokenURI);

function mint(string calldata uri) external returns (uint256 tokenId) {
    require(bytes(uri).length > 0, "URI vide");
    tokenId = _nextTokenId++;            // id unique et croissant
    _safeMint(msg.sender, tokenId);      // le createur devient proprietaire
    _setTokenURI(tokenId, uri);          // lien vers les metadonnees IPFS
    emit NFTMinted(tokenId, msg.sender, uri);
}
```

4.2 Explication ligne par ligne

- **_nextTokenId++** : compteur incrémental. Deux NFT ne peuvent jamais avoir le même id.
- **_safeMint** (et non _mint) : si le destinataire est un contrat, il doit implémenter onERC721Received. Sinon la transaction échoue et le NFT n'est pas perdu dans un contrat qui ne sait pas le gérer.
- **require(uri non vide)** : empêche de créer un NFT sans métadonnées.
- **msg.sender** : c'est toujours l'appelant qui devient propriétaire ; personne ne peut créer un NFT « au nom d'un autre ».
- **L'événement NFTMinted** alimente l'historique du frontend (indexed = recherche rapide par créateur).
- **Les 4 fonctions override** (_update, _increaseBalance, tokenURI, supportsInterface) ne contiennent pas de logique : Solidity exige simplement de préciser quelle extension l'emporte quand deux parents définissent la même fonction.

Choix assumé : pas de Ownable ni de rôles. Le mint est ouvert à tous, comme sur une place de marché publique. Il n'y a donc aucune fonction d'administration à protéger ou à détourner.

5. Le contrat NFTMarketplace (en détail)

Rôle : la salle de vente. Elle ne possède aucun NFT : elle enregistre des annonces et orchestre l'échange entre un vendeur et un acheteur.

5.1 Modèle de données

```
struct Listing {
    address seller;   // address(0) => pas d'annonce active
    uint256 price;    // en wei (1 ETH = 10^18 wei)
}
IERC721Enumerable public immutable nft;        // la collection, fixee au deploiement
mapping(uint256 => Listing) public listings;   // tokenId => annonce
```

Une annonce est identifiée par le tokenId : un NFT ne peut avoir qu'une annonce à la fois, ce qui **rend la double mise en vente impossible par construction**.

5.2 Principe non-custodial

Le NFT **reste chez le vendeur** tant qu'il n'est pas vendu. Le vendeur ne fait qu'**approuver** la marketplace (approve ou setApprovalForAll), c'est-à-dire l'autoriser à transférer ce NFT-là. Avantages : aucun risque que la marketplace garde ou perde un NFT, et le vendeur peut à tout moment retirer l'approbation.

5.3 listNFT : mettre en vente

```
function listNFT(uint256 tokenId, uint256 price) external {
    require(price > 0, "Prix doit etre > 0");
    require(nft.ownerOf(tokenId) == msg.sender, "Vous n'etes pas proprietaire");
    require(nft.getApproved(tokenId) == address(this)
         || nft.isApprovedForAll(msg.sender, address(this)), "Marketplace non approuvee");
    require(listings[tokenId].seller == address(0), "Deja en vente");

    listings[tokenId] = Listing(msg.sender, price);
    emit NFTListed(tokenId, msg.sender, price);
}
```

Verification	Attaque ou erreur évitée
price > 0	Annonce gratuite par erreur
ownerOf == msg.sender	Vendre le NFT d'un autre
approbation de la marketplace	Annonce impossible à honorer (la vente échouerait ensuite)
pas d'annonce existante	Doublon d'annonces

5.4 buyNFT : la fonction critique

```
function buyNFT(uint256 tokenId) external payable nonReentrant {
    Listing memory item = listings[tokenId];
    require(item.seller != address(0), "NFT non en vente");
    require(msg.value == item.price, "Prix incorrect");
    require(msg.sender != item.seller, "Vendeur ne peut pas acheter");

    // 1. EFFETS : on supprime l'annonce AVANT tout appel externe
    delete listings[tokenId];

    // 2. INTERACTIONS : transfert du NFT (revert si plus proprietaire / plus approuve)
    nft.safeTransferFrom(item.seller, msg.sender, tokenId);

    // 3. Paiement du vendeur (revert si echec => le transfert du NFT est annule aussi)
    (bool ok, ) = payable(item.seller).call{value: msg.value}("");
    require(ok, "Paiement echoue");

    emit NFTSold(tokenId, item.seller, msg.sender, item.price);
```

```
}
```

Atomicité : une transaction Ethereum est tout-ou-rien. Si le transfert du NFT échoue, l'ETH de l'acheteur n'est pas débité. Si le paiement du vendeur échoue, le NFT retourne au vendeur. Il est impossible de « payer sans recevoir » ou « recevoir sans payer ».

Protection	Mécanisme
Double vente	L'annonce est supprimée avant les appels externes ; un second acheteur voit « NFT non en vente ».
Réentrance	Modificateur nonReentrant (OpenZeppelin) + schéma Checks-Effects-Interactions : même un contrat vendeur malveillant ne peut pas rappeler buyNFT pendant le paiement.
Mauvais prix	msg.value doit être égal au prix, pas de trop-perçu à gérer ni à rembourser.
Annonce périmée	Si le vendeur a transféré le NFT ou retiré l'approbation, safeTransferFrom échoue et tout est annulé.
Fonds bloqués	Le contrat ne conserve aucun ETH : tout est renvoyé au vendeur dans la même transaction (vérifié par un test : solde du contrat = 0).

5.5 cancelListing et getActiveListings

```
function cancelListing(uint256 tokenId) external {
    require(listings[tokenId].seller == msg.sender, "Vous n'etes pas le vendeur");
    delete listings[tokenId];
    emit ListingCancelled(tokenId, msg.sender);
}
```

- **cancelListing** : réservé au vendeur ; après annulation, l'achat est impossible et le NFT peut être remis en vente.
- **getActiveListings** (fonction de lecture, gratuite) : parcourt la collection et ne renvoie que les annonces **encore valides** (vendeur toujours propriétaire ET approbation toujours active). La page Explorer n'affiche donc jamais d'annonce impossible à acheter.

Limite assumée : getActiveListings boucle sur toute la collection. C'est parfait pour un projet étudiant, mais inadapté à des milliers de NFT. En production on utiliserait un indexeur (The Graph) – voir section 12.

6. Scénario complet d'une vente

#	Acteur	Action	Contrat appelé	Effet
1	Sara	Crée un NFT (image + JSON sur IPFS)	MyNFT.mint(uri)	Sara possède le NFT n°1
2	Sara	Autorise la marketplace	MyNFT.approve(market, 1)	La marketplace peut transférer le n°1
3	Sara	Fixe le prix : 1 ETH	NFTMarketplace.listNFT(1, 1 ETH)	Annonce enregistrée + événement NFTListed
4	Paul	Achète en envoyant 1 ETH	NFTMarketplace.buyNFT(1)	NFT -> Paul, 1 ETH -> Sara, événement NFTSold
5	Sara (option)	Aurait pu annuler avant l'achat	NFTMarketplace.cancelListing (1)	Annonce supprimée

C'est la raison pour laquelle l'interface « Vendre » comporte deux boutons : **1. Approuver** puis **2. Mettre en vente**.

7. Tests automatisés (28 tests, tous réussis)

Les tests sont écrits avec Hardhat, Mocha et Chai. Ils couvrent les cas normaux **et** les cas d'erreur exigés par le sujet.

Cas du sujet	Test correspondant
Mauvais prix	Prix trop bas et trop haut refusés (« Prix incorrect »)
NFT non détenu	Mise en vente par un non-propriétaire refusée
NFT non approuvé	Mise en vente sans approbation refusée
Achat déjà effectué	Second achat refusé (« NFT non en vente »)
Annonce annulée	Achat impossible après annulation ; remise en vente possible
Utilisateur non connecté	Géré côté frontend (composant RequireWallet)
Atomicité	Un vendeur qui refuse l'ETH fait échouer l'achat : le NFT reste chez lui et l'annonce est intacte
Annonce périmée	NFT transféré ou approbation retirée après la mise en vente : l'achat échoue
Flux d'argent	Solde de l'acheteur -prix, solde du vendeur +prix, solde du contrat = 0
Événements	NFTMinted, NFTListed, NFTSold, ListingCancelled émis avec les bons arguments

Un petit contrat de test, `RejectingSeller`, simule un vendeur malveillant ou défaillant. Il n'est jamais déployé en production.

Ce qui n'est pas testé : pas de test de réentrance avec un contrat attaquant dédié (la protection repose sur nonReentrant et l'ordre des opérations) ; pas encore de test automatisé du frontend (étape 9).

8. Déploiement

Le script `scripts/deploy.js` :

- déploie MyNFT, puis NFTMarketplace en lui passant l'adresse de MyNFT (d'où cet ordre) ;
- lit le chainId du réseau (31337 en local, 11155111 sur Sepolia) ;
- écrit les adresses **par réseau** dans `frontend/src/contracts/deployments.json` et copie les ABI : le frontend n'a rien à recopier à la main et choisit les bonnes adresses selon le réseau MetaMask.

```
npx hardhat node                                        # terminal 1 : blockchain locale
npx hardhat run scripts/deploy.js --network localhost   # terminal 2 : deploiement
cd frontend && npm run dev                              # terminal 3 : application
```

Les adresses de contrats changent à chaque redémarrage du nœud local (la chaîne repart de zéro) : il faut alors relancer le déploiement.

9. Frontend React

Page / module	Fonction
Explorer	Galerie des NFT en vente (image, nom, prix, vendeur) et bouton Acheter. Le vendeur voit « Votre annonce » au lieu du bouton.
Créer	Formulaire : image, nom, description. Validations (type PNG/JPEG/GIF/WebP, taille, longueurs), upload IPFS puis mint.
Vendre	Choix d'un NFT possédé, bouton Approuver, saisie du prix validée, mise en vente.
Mes NFT	NFT détenus, état (en vente ou non), annulation d'annonce, historique reconstruit à partir des événements on-chain.
Wallet (useWallet)	Connexion/déconnexion, adresse, détection du réseau, bouton de changement de réseau, gestion d'erreurs (MetaMask absent, demande refusée, demande déjà ouverte).
useTx	Suit chaque transaction : signature en attente, envoyée, confirmée ou en erreur, avec message lisible.

Points de conception : le composant RequireWallet bloque toute page si MetaMask est absent, non connecté ou sur un mauvais réseau ; les erreurs techniques d'ethers/MetaMask sont traduites en messages compréhensibles ; avant chaque opération l'application vérifie que les contrats existent bien sur la chaîne (utile après un redémarrage du nœud local).

10. Stockage IPFS et gestion des secrets

Pour créer un NFT, l'application envoie d'abord l'image à Pinata (récupère un CID), construit le JSON de métadonnées `{name, description, image: ipfs://CID}`, l'envoie aussi, puis enregistre `ipfs://CID_du_JSON` dans le contrat via mint.

Sécurité des secrets : une variable VITE_xxx est incluse dans le code envoyé au navigateur, donc visible par tous. La clé Pinata (JWT) est donc placée dans frontend/.env **sans préfixe VITE_** et n'est lue que par le serveur Vite, dont le proxy ajoute l'en-tête d'autorisation. Le navigateur ne la voit jamais. Les fichiers .env sont exclus de git.

Limite : ce proxy n'existe que pendant npm run dev / vite preview. Un hébergement public nécessiterait un petit backend chargé d'ajouter la clé.

11. Synthèse sécurité

Exigence du sujet	Réponse dans le projet
Standards OpenZeppelin	ERC721, URIStorage, Enumerable, ReentrancyGuard
Permissions et propriété	ownerOf et approbation vérifiés à la mise en vente et à l'achat ; annulation réservée au vendeur
Réentrance et erreurs de paiement	nonReentrant, Checks-Effects-Interactions, paiement vérifié avec require
Événements	NFTMinted, NFTListed, NFTSold, ListingCancelled
Validations frontend et contrats	Les deux couches valident : le frontend pour l'ergonomie, les contrats pour la sécurité réelle
Pas de secret exposé	JWT Pinata côté serveur, .env ignoré par git, clés de test Hardhat uniquement en local

12. Limites et améliorations possibles

Limite actuelle	Amélioration possible
Pas d'enchères, uniquement prix fixe	Ajouter enchères ou offres
Aucune commission de plateforme	Frais en pourcentage (avec plafond) pour le propriétaire de la marketplace
Pas de royalties pour le créateur	Standard ERC-2981 pour reverser un pourcentage à chaque revente
Pas de modification du prix d'une annonce	Fonction updatePrice (aujourd'hui : annuler puis relister)
getActiveListings parcourt toute la collection	Indexeur (The Graph) ou pagination
Une seule collection (MyNFT)	Marketplace multi-collections avec adresse du contrat dans l'annonce
Paiement direct au vendeur	Modèle « retrait » (pull payment) pour éliminer le risque d'un vendeur-contrat qui bloque sa propre vente
Contrats non audités	Audit externe, analyse statique (Slither), tests de fuzzing
Proxy Pinata limité au mode développement	Petit backend ou fonction serverless pour la production

13. Guide de présentation orale

Plan suggéré (8 à 10 minutes)

- **Problème (1 min)** : acheter/vendre des objets numériques uniques sans tiers de confiance.
- **Solution (1 min)** : deux contrats, un pour la propriété, un pour la vente.
- **Démo (3 min)** : créer, mettre en vente, acheter avec un second compte, montrer l'historique.
- **Zoom contrats (3 min)** : non-custodial, atomicité de buyNFT, protection contre la double vente et la réentrance.
- **Qualité et limites (1 min)** : 28 tests, limites honnêtes, améliorations.

Questions probables du jury

Question	Réponse courte
Pourquoi ne pas mettre l'image sur la blockchain ?	Coût de gas prohibitif ; on stocke l'empreinte (CID) et l'image sur IPFS.
Que se passe-t-il si le paiement échoue ?	Toute la transaction est annulée, y compris le transfert du NFT : l'atomicité d'Ethereum.
Comment éviter la double vente ?	L'annonce est supprimée avant tout appel externe et nonReentrant bloque les rappels.
La marketplace peut-elle voler mes NFT ?	Non : elle ne les détient pas, et elle ne peut transférer que ceux que j'ai approuvés, uniquement via buyNFT et au prix annoncé.
Pourquoi une approbation avant la vente ?	C'est le mécanisme ERC-721 pour déléguer un transfert sans céder la propriété.
Un vendeur peut-il changer le prix après la mise en vente ?	Pas directement : il doit annuler puis relister (amélioration prévue : updatePrice).
Pourquoi le mint est-il ouvert à tous ?	C'est le comportement d'une place de marché publique ; aucune fonction d'administration à attaquer.
Et si l'image disparaît d'IPFS ?	Le contenu reste tant qu'il est « pinné » (Pinata). Sans pinning, un nœud peut l'oublier : risque propre à IPFS.

14. Suite du projet

- **Étape 9** : test d'intégration automatisé du parcours complet + checklist de tests manuels avec MetaMask.
- **Étape 10** : mesures de latence (envoi vs confirmation), gas, frais estimés (gasUsed x effectiveGasPrice) et débit, en local puis sur Sepolia.
- **Étape 11** : déploiement Sepolia et README final.